

GUÍA DE ROLES

Reglas a seguir para cada rol existente en el equipo.

CONTEXTO

Para que haya un orden en el equipo, se necesita un manual que nos guíe en la creación de cualquier proyecto, que nos diga qué sí y qué no hacer. Esto debe de ser un estándar para todos los desarrolladores presentes para que el proyecto se desarrolle de manera uniforme.

La regla de oro es que TODOS los desarrolladores sigan al pie de la letra la guía para su rol específico. ¿Por qué es esto necesario? En nuestro proyecto, al estar compuesto de seis integrantes, necesitamos trabajar de una misma manera para mantener la uniformidad en el transcurso de todo el proyecto y evitar cualquier tipo de ambigüedad o problema futuro interno que se pueda presentar. Es por eso que se desarrolló esta guía. Por favor, **lea con atención su rol**.

1. EL ARQUITECTO

La función del Arquitecto no es sólo dar las ideas a desarrollar, sino que guiar paso a paso a los **Programadores** y, en algunos casos, **Testers**, explicando explícitamente y sin ambigüedad qué es lo que se quiere lograr con la nueva versión del proyecto, cómo se tiene pensado lograrlo y explicar conceptos complejos que usted, como Arquitecto, podría necesitar para la versión en desarrollo. Sólo debe de desarrollar el molde a utilizar, no la pieza final.

Guía de Criterios

1. Siempre utilizar la plantilla “Juego, Plantilla para Arquitecto” para el desarrollo de la nueva versión.
2. Definir claramente y sin ambigüedad qué es lo que se quiere lograr en la nueva versión a desarrollar. Prioriza utilizar un lenguaje simple.
3. Explicar con lujo de detalle la serie de pasos que necesitan seguir los programadores para desarrollar las nuevas funciones. Puedes usar series de **Pasos a Seguir, Diagramas de Flujo** o cualquier otro tipo de diagrama que domines.
4. Para la creación de funciones o clases, explica qué es lo que se necesita para implementarlas, qué devuelven y su propósito para cada una nueva creada.
5. Tienes la responsabilidad de dar el visto bueno al programa desarrollado por los programadores antes de pasar a los **Testers**.

2. EL PROGRAMADOR

El Programador es el que realmente se encarga de dar vida a las palabras del Arquitecto. Su objetivo es tan simple como implementar lo pedido para la nueva versión siguiendo los pasos brindados en el documento entregado por el Arquitecto.

Guía de Criterios

1. **PROHIBIDO** utilizar la herramienta **using namespace std** dentro del programa del juego. Esta herramienta te permite recortar la nomenclatura estándar **std** (simplemente significa "Standard") para todas las funciones dentro de las librerías de C++. Se bloquea su uso para eliminar posibles ambigüedades con otras funciones creadas dentro del programa y evitar errores futuros.
2. Utilizar, para líneas de código anidadas, sangrías bien definidas para priorizar un código limpio y legible para cualquier programador.
3. Utilizar lo más posible los comentarios (**//** o **/**/**) para no sólo decir qué hace cierta clase o función, pero para explicar por qué se creó de la manera en la que se hizo y cuál fue su propósito original.
4. Mantener un nombre de variables siempre claro. Sé exageradamente detallado con los nombres; que éste te diga para qué sirve y dónde va casa cosa sin necesidad de buscar comentarios. Evita lo más posible las variables, funciones y clases de una sola letra o palabras ambiguas.
5. Siempre consultar al **Arquitecto** en caso de alguna duda sobre el documento.
6. Al finalizar el trabajo, tiene la obligación de mostrar el resultado al **Arquitecto** antes de pasar el trabajo a los **Testers**.

3. EL TESTER

El propósito del Tester no es sólo probar el programa y ver que funcione, es hacer un control de calidad sobre cada una de las nuevas funciones agregadas y explotarlas, buscando romper el programa a propósito para ver qué falla y qué no.

Guía de Criterios

1. En cada versión, tiene la responsabilidad de ver en el documento compartido por el Arquitecto cuáles fueron las nuevas funciones agregadas para saber qué tiene prioridad al probar.
2. Siempre buscar errores en todos los lados posibles; introduce datos erróneos, repite una misma acción miles de veces o trata de hacer cosas fuera de lo convencional para ver qué se rompe y qué funciona.
3. Cuando encuentre un error, tiene la responsabilidad de documentar en una lista de bugs qué fue lo que hizo, en qué parte y por qué se rompió. No tiene que dar la solución al problema, simplemente debe de hacer consciencia de su existencia.
4. Compartir la documentación obtenida, desarrollada con un lenguaje simple, a todo el grupo para que los futuros Arquitectos, Programadores y Testers sepan en qué enfocarse en la próxima versión.

4. EL DOCUMENTADOR

En resumidas cuentas, es el ángel de la guarda del equipo. Su responsabilidad es guardar el progreso llevado a cabo desde la primera versión hasta la final, siempre contando cuales fueron los cambios realizados en cada nueva versión que se agregue mediante un "readme.txt".

Guía de Criterios

1. Subir todos los archivos compartidos en el lapso de tiempo trabajado desde la última versión hasta la actual en el repositorio de **GitHub**. Esto no incluye el programa **Juego.cpp**, los documentos compartidos por el Arquitecto o la documentación de los Testers.
2. Crear un archivo **readme.txt** que detalle todos los nuevos cambios realizados en el programa. Tienen que estar enlistados y divididos por secciones bien definidas: por ejemplo, una sección llamada Clases que especifique sólo las nuevas Clases creadas y su función, si hubieron modificaciones, si se eliminaron algunas, etc.; lo mismo podría realizarse en una sección para Funciones o cualquier cosa que se vea necesaria. Esto depende totalmente del Documentador.
3. Tiene la obligación de contarle a todo el equipo mediante el grupo de WhatsApp qué modificaciones se realizaron dentro del repositorio.
4. Al seguir el ciclo de cambio, tendrá que contarle al equipo que ahora ha asumido el rol de Arquitecto una vez haya terminado su trabajo de Documentador.